ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA



BÁO CÁO BÀI TẬP LỚN SỐ 2

HỆ THỐNG QUẢN LÝ HỌC TẬP (LMS)

Phân tích, thiết kế, cài đặt và kiểm thử cơ sở dữ liệu

Môn học: Hệ Cơ sở dữ liệu

Mã môn học: CO3094

Nhóm thực hiện:	Nhóm BTL2
Thành viên:	TV1: [Họ tên] – [MSSV]
	TV2: [Họ tên] – [MSSV]
	TV3: [Họ tên] – [MSSV]
	TV4: [Họ tên] – [MSSV]
	TV5: [Họ tên] – [MSSV]
Giảng viên hướng dẫn:	[Điền tên giảng viên]
Khoa:	Khoa Khoa học và Kỹ thuật Máy tính

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

1 Giới thiệu **1**

1.1 Bối cảnh và mục tiêu . . . 1

1.2 Phạm vi và cách tiếp cận . . . 1

1.3 Kết quả đạt được . . . 2

1.4 Cấu trúc báo cáo . . . 2

2 Quy trình thực hiện **2**

2.1 Khảo sát và phân tích nghiệp vụ . . . 2

2.2 Thiết kế và chuẩn hóa . . . 3

2.3 Cài đặt và kiểm thử . . . 3

2.4 Tiếp nhận phản hồi và sửa lỗi . . . 3

3 Thiết kế cơ sở dữ liệu **3**

3.1 Mô hình tổng quan . . . 3

3.2 Kiểm kê 22 bảng, cột, khóa ngoại và CHECK . . . 4

3.3 Các quyết định thiết kế quan trọng . . . 9

3.3.1 ISA dùng chung khóa chính . . . 9

3.3.2 Bảng nối và thực thể yếu . . . 10

3.3.3 Thuộc tính dẫn xuất . . . 10

3.3.4 Phân tách ràng buộc . . . 10

3.4 Quy tắc nghiệp vụ và nơi cài đặt . . . 10

4 Trigger **11**

4.1 Vai trò và nguyên tắc . . . 11

4.2 Danh sách 13 trigger . . . 12

4.3 Trigger trùng lịch RB05 . . . 12

4.4 Trigger thuộc tính dẫn xuất . . . 13

4.5 Ma trận DML . . . 13

4.6 Mã tiêu biểu . . . 13

5 Stored procedure **14**

5.1 Danh sách procedure . . . 14

5.2 CRUD trên `Enrollment` . . . 14

5.3 Procedure truy vấn . . . 15

5.4 Luồng UI – API – procedure . . . 15

5.5 Mã tiêu biểu 15

6 Stored function **16**

6.1 Yêu cầu và cách đáp ứng 16

6.2 `fn_CalculateAssignmentCompletionRate` 17

6.3 `fn_CalculateStudentAverageScore` 17

6.4 Kết quả kiểm thử 17

7 Ứng dụng **18**

7.1 Kiến trúc 18

7.2 Phân quyền theo vai trò 18

7.3 Giao diện 19

7.4 Luồng CRUD Ghi danh 23

7.5 Cấu hình và chạy 23

8 Kiểm thử **24**

8.1 Chiến lược kiểm thử 24

8.2 Kiểm thử CSDL 24

8.3 Kiểm thử Python 25

8.4 Smoke API và phân quyền 25

8.5 Kiểm thử trình duyệt 26

8.6 Readiness và việc còn lại 26

9 Phân công, kết luận và hạn chế **27**

9.1 Phân công thành viên 27

9.2 Kết luận 27

9.3 Hạn chế và hướng phát triển 28

A Mã schema tiêu biểu **29**

B Mã trigger tiêu biểu **31**

C Mã procedure tiêu biểu **32**

D Mã function **34**

E Lệnh kiểm tra và checklist nộp bài **37**

Danh sách bảng

1 Tóm tắt kết quả định lượng của bài tập . . . 2
2 Danh mục 22 bảng của cơ sở dữ liệu . . . 4
3 Danh sách cột của từng bảng (119 cột) . . . 5
4 35 khóa ngoại và quy tắc xóa/cập nhật . . . 6
5 39 ràng buộc CHECK theo miền giá trị và ngữ nghĩa . . . 7
6 15 chỉ mục hỗ trợ lọc và join . . . 8
7 Số hàng seed của 22 bảng (mọi bảng đạt ≥ 5) . . . 9
8 Ánh xạ 17 quy tắc nghiệp vụ sang cơ chế cài đặt . . . 10
9 13 trigger và phạm vi bảo vệ . . . 12
10 Ma trận DML được bảo vệ bởi trigger . . . 13
11 5 stored procedure và mục đích . . . 14
12 Đối chiếu yêu cầu của hai procedure truy vấn . . . 15
13 Đối chiếu yêu cầu rubric của hai function . . . 16
14 Kết quả gọi hàm trên dữ liệu mẫu . . . 17
15 Nhóm API và vai trò yêu cầu . . . 19
16 Luồng UI – API – procedure của CRUD Ghi danh . . . 23
17 Các tầng kiểm thử và phạm vi . . . 24
18 Các nhóm test Python . . . 25
19 Kiểm tra 401/403/200 trên API quản trị . . . 26
20 Phân công 5 thành viên và object sở hữu . . . 27
21 Hạn chế đã biết và hướng khắc phục . . . 28

Danh sách hình vẽ

1 Quy trình tám bước từ mô hình BTL1 đến ứng dụng và kiểm thử 3
2 Nhóm bảng theo miền dữ liệu và quan hệ chính 4
3 Đường đi của một thao tác CRUD Ghi danh 15
4 Kiến trúc bốn lớp của ứng dụng 18
5 Màn hình chọn vai trò và tài khoản demo 20
6 Dashboard sinh viên 20
7 Danh sách lớp học phần của sinh viên 21
8 Tab Quản trị → Ghi danh với bộ lọc và CRUD 22
9 Tab Quản trị → Thống kê gọi procedure và function 23

1. Giới thiệu

1.1 Bối cảnh và mục tiêu

Báo cáo này trình bày bài tập lớn số 2 của môn Hệ Cơ sở dữ liệu (CO3094), tiếp nối bài tập lớn số 1 đã phân tích nghiệp vụ và thiết kế mô hình dữ liệu cho một hệ thống quản lý học tập (Learning Management System – LMS) theo mô hình của Trường Đại học Bách khoa – ĐHQG TP. Hồ Chí Minh. BTL1 đã chốt 20 thực thể, 2 quan hệ ISA và 28 mối quan hệ, kèm 17 quy tắc nghiệp vụ RB01–RB17. BTL2 yêu cầu hiện thực hóa mô hình đó thành một cơ sở dữ liệu quan hệ chạy được, có dữ liệu mẫu, trigger, thủ tục, hàm, và một ứng dụng thực sự gọi các đối tượng này.

Mục tiêu cụ thể:

- Cài đặt đầy đủ 22 bảng quan hệ, khóa chính, khóa ngoại, ràng buộc miền giá trị và chỉ mục cần thiết.
- Sinh dữ liệu mẫu có ý nghĩa cho mọi bảng, tối thiểu 5 hàng mỗi bảng.
- Cài đặt 3 procedure CRUD trên bảng `Enrollment`, 2 procedure truy vấn, 2 function có cursor/IF/LOOP, và 13 trigger cho các quy tắc liên bảng.
- Xây dựng ứng dụng web Flask + giao diện SPA, phân quyền theo vai trò Student/Staff/Admin, gọi thật procedure/function từ màn hình.
- Kiểm thử tự động và kiểm thử đầu-cuối, ghi lại bằng chứng trung thực.

1.2 Phạm vi và cách tiếp cận

Phạm vi dữ liệu bám theo BTL1: tài khoản, khoa, sinh viên, giảng viên, quản trị viên, học kỳ, môn học, thành phần môn học, lớp học phần, ghi danh, thông báo, tài nguyên, đánh giá, bài tập, bài nộp, quiz, điểm và phản hồi. Ứng dụng chỉ phục vụ mục đích minh họa học thuật, không phải hệ thống production.

Cách tiếp cận gồm bốn lớp: (1) tầng CSDL MySQL với schema, constraint, trigger, procedure, function; (2) tầng truy cập dữ liệu Python dùng PyMySQL; (3) tầng dịch vụ và API Flask; (4) tầng giao diện SPA offline dùng Bootstrap đã vendored. Mọi thao tác ghi `Enrollment` trên giao diện đều đi qua procedure, không dùng câu lệnh `INSERT/UPDATE/DELETE` trực tiếp.

1.3 Kết quả đạt được

Bảng 1: Tóm tắt kết quả định lượng của bài tập

Hạng mục	Kết quả
Bảng quan hệ	22 bảng, 119 cột
Khóa ngoại / UNIQUE / CHECK	35 FK, 10 UNIQUE, 39 CHECK
Chỉ mục hỗ trợ	15 index
Trigger	13 trigger
Stored procedure	5 procedure
Stored function	2 function
Dữ liệu mẫu	22/22 bảng có ≥ 5 hàng
Kiểm thử Python	72 passed
Kiểm thử CSDL	`DATABASE VERIFICATION: PASS`
Smoke API thật	31/31 PASS
Kiểm thử trình duyệt	Chromium headless, không có lỗi console

1.4 Cấu trúc báo cáo

Chương 2 mô tả quy trình thực hiện. Chương 3 trình bày thiết kế CSDL và kiểm kê đầy đủ các bảng. Chương 4, 5, 6 lần lượt trình bày trigger, procedure và function. Chương 7 mô tả ứng dụng, phân quyền và giao diện. Chương 8 tổng hợp kiểm thử. Chương 9 là phân công, kết luận và hạn chế. Phần phụ lục chứa mã SQL tiêu biểu và các lệnh kiểm tra.

2. Quy trình thực hiện

2.1 Khảo sát và phân tích nghiệp vụ

Nhóm kế thừa kết quả BTL1: mô hình ERD, 20 thực thể, 2 quan hệ ISA (`Student`/`Staff`/`Admin` là các `UserAccount`), 28 mối quan hệ và 17 quy tắc RB01–RB17. Các quy tắc được phân loại theo nơi cài đặt:

- **CHECK/UNIQUE/FK**: miền giá trị, duy nhất, tham chiếu.
- **Trigger**: quy tắc cần đọc nhiều hàng hoặc nhiều bảng, ví dụ trùng lịch, sĩ số, thuộc tính dẫn xuất `Assignment.SubmissionCount`.

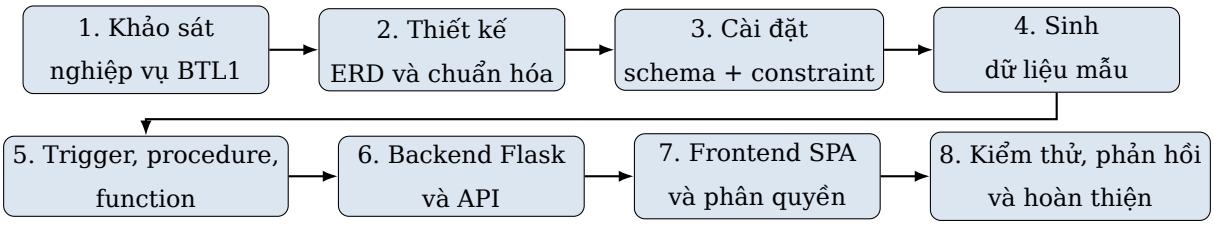
- **Procedure**: nghiệp vụ có nhiều bước và cần transaction, ví dụ thêm ghi danh phải kiểm tra tài khoản, lớp, sĩ số, trùng lặp.
- **Application/service**: phân quyền, ownership, kiểm tra dữ liệu đầu vào và các quy tắc trình bày.

2.2 Thiết kế và chuẩn hóa

Từ ERD, nhóm ánh xạ quan hệ 1:1, 1:N và N:M sang bảng. Các quan hệ ISA được cài đặt bằng bảng cha `UserAccount` và ba bảng con dùng chung khóa chính. Quan hệ N:M `CourseSection–LearningResource` được cài đặt bằng bảng nối `Section_Resource`. Các thực thể yếu như `SubmissionFile` dùng khóa chính ghép. Thiết kế đạt dạng chuẩn 3 (3NF): mỗi bảng có khóa chính rõ ràng, thuộc tính không khóa phụ thuộc đầy đủ vào khóa, không có phụ thuộc bắc cầu.

2.3 Cài đặt và kiểm thử

Thứ tự chạy CSDL được cố định: tạo database, tạo bảng, thêm khóa ngoại, thêm CHECK, tạo index, tạo trigger, procedure, function, seed, rồi verify. Việc tách giai đoạn giúp phát hiện lỗi tham chiếu sớm và cho phép chạy lại idempotent. Ứng dụng được cài đặt song song với các test tầng repository để chứng minh API gọi đúng procedure.



Hình 1: Quy trình tám bước từ mô hình BTL1 đến ứng dụng và kiểm thử

2.4 Tiếp nhận phản hồi và sửa lỗi

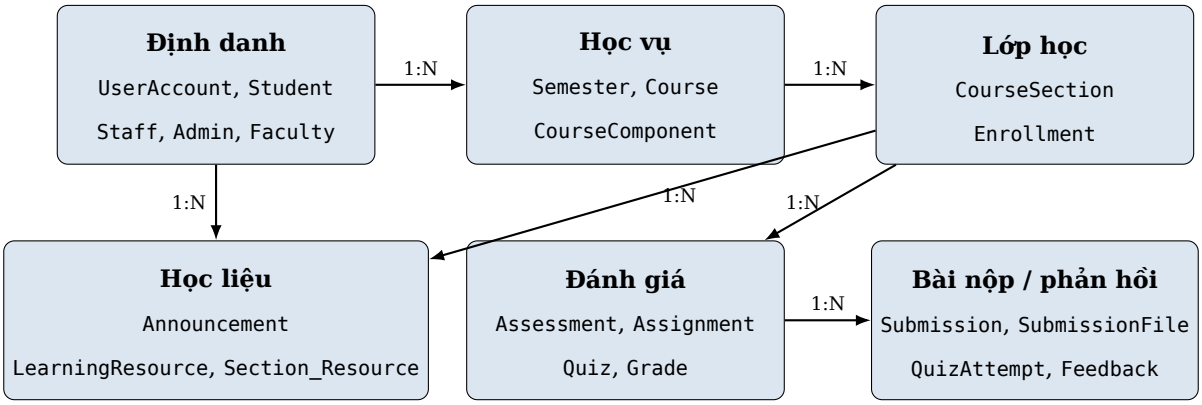
Sau khi có bản chạy đầu, nhóm rà lại theo rubric và phản hồi của người đánh giá. Các lỗi thực tế đã sửa gồm: ánh xạ `StudentCode` sai bảng, `DayOfWeek` lệch một đơn vị, thuộc tính HTML bị hỏng, thiếu tên tệp đính kèm, nút nộp bài vẫn hiện khi hết lượt, tạo tài khoản Staff thiếu `faculty_id`, và API quản trị chưa được bảo vệ. Các thay đổi được ghi trong lịch sử Git với tiền tố `feat`, `fix`, `test`, `docs` và tham chiếu rubric.

3. Thiết kế cơ sở dữ liệu

3.1 Mô hình tổng quan

Hình 2 nhóm 22 bảng thành năm miền: định danh, học vụ, lớp học, học liệu và đánh giá. Cách nhóm này phản ánh đường đi của dữ liệu trong ứng dụng: một tài khoản thuộc một

khoa; một sinh viên ghi danh vào lớp học phần; lớp có thông báo, tài nguyên và đánh giá; đánh giá sinh ra bài tập/quiz, bài nộp, điểm và phản hồi.



Hình 2: Nhóm bảng theo miền dữ liệu và quan hệ chính

3.2 Kiểm kê 22 bảng, cột, khóa ngoại và CHECK

Các bảng dưới đây được sinh trực tiếp từ `database/schema/01_tables.sql`, `database/constraints/02_foreign_keys.sql`, `database/constraints/03_checks.sql`, `database/indexes/04_indexes.sql` và `docs/ROW_COUNTS.md`. Nhờ vậy, số cột, khóa và biểu thức ràng buộc khớp với CSDL thật thay vì chép tay.

Bảng 2: Danh mục 22 bảng của cơ sở dữ liệu

#	Bảng	Cột	PK	FK	UQ	CK	Mục đích
1	`UserAccount`	6	`UserID`	0	1	3	Tài khoản dùng chung; lưu email, họ tên, avatar, trạng thái, lần truy cập cuối.
2	`Faculty`	3	`FacultyID`	0	0	1	Danh mục khoa.
3	`Student`	3	`UserID`	2	1	1	Hồ sơ sinh viên (ISA của UserAccount) và khoa trực thuộc.
4	`Staff`	2	`UserID`	2	0	0	Hồ sơ giảng viên/cán bộ (ISA của UserAccount) và khoa.
5	`Admin`	1	`UserID`	1	0	0	Hồ sơ quản trị viên (ISA của UserAccount).
6	`Semester`	5	`SemesterID`	0	1	3	Học kỳ theo năm học và số học kỳ.
7	`Course`	6	`CourseID`	2	1	2	Môn học, số tín chỉ, khoa quản lý, người tạo.
8	`CourseComponent`	4	`ComponentID`	1	1	1	Thành phần môn học: lý thuyết hoặc thực hành/bài tập.
9	`CourseSection`	11	`SectionID`	3	1	5	Lớp học phần cụ thể trong học kỳ: lịch, phòng, sĩ số, giảng viên.

#	Bảng	Cột	PK	FK	UQ	CK	Mục đích
10	Enrollment	5	EnrollmentID	2	1	1	Ghi danh sinh viên vào lớp học phần; bảng trung tâm của CRUD.
11	Announcement	7	AnnouncementID	2	0	1	Thông báo của lớp học phần.
12	LearningResource	7	ResourceID	1	0	2	Tài nguyên học tập dùng chung.
13	Section_Resource	5	SectionID, ResourceID	2	0	1	Liên kết N:M giữa lớp học phần và tài nguyên.
14	Assessment	8	AssessmentID	2	0	2	Đánh giá chung của lớp: assignment/quiz, điểm tối đa, số lần làm.
15	Assignment	5	AssessmentID	1	0	2	Bài tập thuộc assessment; hạn nộp và số bài đã nộp (derived).
16	AssignmentAttachment	2	AssessmentID, AttachmentURL	1	0	1	Tệp đính kèm của bài tập (metadata).
17	Submission	9	SubmissionID	3	1	4	Bài nộp của sinh viên cho assignment.
18	SubmissionFile	6	SubmissionID, FileNo	1	0	2	Tệp trong một bài nộp (metadata).
19	Quiz	4	AssessmentID	1	0	2	Bài kiểm tra trắc nghiệm thuộc assessment.
20	QuizAttempt	7	AttemptID	2	1	3	Một lần làm quiz của sinh viên.
21	Grade	7	GradeID	3	1	1	Điểm của một enrollment cho một assessment.
22	Feedback	6	FeedbackID	3	0	1	Nhận xét, có thể trả lời lồng nhau.

Bảng 3: Danh sách cột của từng bảng (119 cột)

Bảng	Các cột
UserAccount	UserID, Email, FullName, AvatarURL, AccountStatus, LastAccessAt
Faculty	FacultyID, FacultyName, FacultyNameEN
Student	UserID, StudentCode, FacultyID
Staff	UserID, FacultyID
Admin	UserID
Semester	SemesterID, AcademicYear, TermNo, StartDate, EndDate
Course	CourseID, CourseCode, CourseName, Credits, FacultyID, CreatedByAdminID
CourseComponent	ComponentID, CourseID, ComponentType, ComponentName
CourseSection	SectionID, ComponentID, SemesterID, PrimaryInstructorID, SectionCode, Capacity, SectionStatus, DayOfWeek, StartTime, EndTime, RoomCode

Bảng	Các cột
Enrollment	EnrollmentID, StudentID, SectionID, EnrollmentDate, EnrollmentStatus
Announcement	AnnouncementID, SectionID, PostedByStaffID, Title, Content, PostedAt, ExpiredAt
LearningResource	ResourceID, UploadedByStaffID, Title, Description, ResourceType, ResourceURL, UploadedAt
Section_Resource	SectionID, ResourceID, PostedAt, DisplayOrder, IsVisible
Assessment	AssessmentID, SectionID, CreatedByStaffID, Title, Description, CreatedAt, MaxScore, MaxAttempts
Assignment	AssessmentID, OpenDate, DueDate, AllowLateSubmission, SubmissionCount
AssignmentAttachment	AssessmentID, AttachmentURL
Submission	SubmissionID, AssignmentID, StudentID, GradedByStaffID, AttemptNo, SubmittedAt, SubmissionStatus, Score, GradedAt
SubmissionFile	SubmissionID, FileNo, FileName, FileURL, FileSize, UploadedAt
Quiz	AssessmentID, OpenTime, CloseTime, DurationMinutes
QuizAttempt	AttemptID, QuizID, StudentID, AttemptNo, StartedAt, FinishedAt, Score
Grade	GradeID, EnrollmentID, AssessmentID, GradedByStaffID, Score, GradedAt, IsPublished
Feedback	FeedbackID, SubmissionID, AuthorID, ParentFeedbackID, Content, CreatedAt

Bảng 4: 35 khóa ngoại và quy tắc xóa/cập nhật

Bảng con	Cột	Bảng cha	Cột cha	Constraint	ON DELETE
Student	UserID	UserAccount	UserID	fk_student_user	CASCADE
Student	FacultyID	Faculty	FacultyID	fk_student_faculty	RESTRICT
Staff	UserID	UserAccount	UserID	fk_staff_user	CASCADE
Staff	FacultyID	Faculty	FacultyID	fk_staff_faculty	RESTRICT
Admin	UserID	UserAccount	UserID	fk_admin_user	CASCADE
Course	FacultyID	Faculty	FacultyID	fk_course_faculty	RESTRICT
Course	CreatedByAdminID	Admin	UserID	fk_course_admin	RESTRICT
CourseComponent	CourseID	Course	CourseID	fk_component_course	CASCADE
CourseSection	ComponentID	CourseComponent	ComponentID	fk_section_component	RESTRICT
CourseSection	SemesterID	Semester	SemesterID	fk_section_semester	RESTRICT
CourseSection	PrimaryInstructorID	Staff	UserID	fk_section_instructor	SET NULL
Enrollment	StudentID	Student	UserID	fk_enrollment_student	RESTRICT
Enrollment	SectionID	CourseSection	SectionID	fk_enrollment_section	RESTRICT
Announcement	SectionID	CourseSection	SectionID	fk_announcement_section	RESTRICT
Announcement	PostedByStaffID	Staff	UserID	fk_announcement_staff	RESTRICT
LearningResource	UploadedByStaffID	Staff	UserID	fk_resource_staff	RESTRICT
Section_Resource	SectionID	CourseSection	SectionID	fk_sr_section	CASCADE
Section_Resource	ResourceID	LearningResource	ResourceID	fk_sr_resource	CASCADE

Bảng con	Cột	Bảng cha	Cột cha	Constraint	ON DELETE
Assessment	SectionID	CourseSection	SectionID	fk_assessment_section	RESTRICT
Assessment	CreatedByStaffID	Staff	UserID	fk_assessment_staff	RESTRICT
Assignment	AssessmentID	Assessment	AssessmentID	fk_assignment_assessment	CASCADE
AssignmentAttachment	AssessmentID	Assignment	AssessmentID	fk_attachment_assignment	CASCADE
Submission	AssignmentID	Assignment	AssessmentID	fk_submission_assignment	RESTRICT
Submission	StudentID	Student	UserID	fk_submission_student	RESTRICT
Submission	GradedByStaffID	Staff	UserID	fk_submission_staff	SET NULL
SubmissionFile	SubmissionID	Submission	SubmissionID	fk_file_submission	CASCADE
Quiz	AssessmentID	Assessment	AssessmentID	fk_quiz_assessment	CASCADE
QuizAttempt	QuizID	Quiz	AssessmentID	fk_attempt_quiz	RESTRICT
QuizAttempt	StudentID	Student	UserID	fk_attempt_student	RESTRICT
Grade	EnrollmentID	Enrollment	EnrollmentID	fk_grade_enrollment	RESTRICT
Grade	AssessmentID	Assessment	AssessmentID	fk_grade_assessment	RESTRICT
Grade	GradedByStaffID	Staff	UserID	fk_grade_staff	RESTRICT
Feedback	SubmissionID	Submission	SubmissionID	fk_feedback_submission	CASCADE
Feedback	AuthorID	UserAccount	UserID	fk_feedback_author	RESTRICT
Feedback	ParentFeedbackID	Feedback	FeedbackID	fk_feedback_parent	SET NULL

Bảng 5: 39 ràng buộc CHECK theo miền giá trị và ngữ nghĩa

Bảng	Constraint	Biểu thức
UserAccount	ck_user_status	`AccountStatus` IN ('ACTIVE', 'LOCKED', 'DISABLED')
UserAccount	ck_user_email	`Email` LIKE '%_@_%._%' AND CHAR_LENGTH(`FullName`) > 0
UserAccount	ck_user_avatar	CHAR_LENGTH(`AvatarURL`) > 0
Faculty	ck_faculty_name	CHAR_LENGTH(`FacultyName`) > 0
Student	ck_student_code	CHAR_LENGTH(`StudentCode`) BETWEEN 4 AND 20
Semester	ck_semester_term	`TermNo` BETWEEN 1 AND 3
Semester	ck_semester_dates	`StartDate` < `EndDate`
Semester	ck_semester_year	`AcademicYear` REGEXP '^[0-9]{4}-[0-9]{4}$'
Course	ck_course_credits	`Credits` BETWEEN 1 AND 10
Course	ck_course_code	CHAR_LENGTH(`CourseCode`) > 0
CourseComponent	ck_component_type	`ComponentType` IN ('THEORY', 'PRACTICE')
CourseSection	ck_section_capacity	`Capacity` > 0
CourseSection	ck_section_status	`SectionStatus` IN ('OPEN', 'CLOSED', 'CANCELLED')
CourseSection	ck_section_day	`DayOfWeek` BETWEEN 1 AND 7
CourseSection	ck_section_time	`StartTime` < `EndTime`
CourseSection	ck_section_room	CHAR_LENGTH(`RoomCode`) > 0
Enrollment	ck_enrollment_status	`EnrollmentStatus` IN ('ACTIVE', 'DROPPED', 'COMPLETED')

Bảng	Constraint	Biểu thức
Announcement	ck_announcement_expiry	`ExpiredAt` IS NULL OR `ExpiredAt` > `PostedAt`
LearningResource	ck_resource_type	`ResourceType` IN ('DOCUMENT', 'SLIDE', 'VIDEO', 'LINK')
LearningResource	ck_resource_url	CHAR_LENGTH(`ResourceURL`) > 0
Section_Resource	ck_sr_order	`DisplayOrder` >= 0
Assessment	ck_assessment_score	`MaxScore` > 0
Assessment	ck_assessment_attempts	`MaxAttempts` IS NULL OR `MaxAttempts` > 0
Assignment	ck_assignment_period	`OpenDate` < `DueDate`
Assignment	ck_assignment_count	`SubmissionCount` >= 0
AssignmentAttachment	ck_attachment_url	CHAR_LENGTH(`AttachmentURL`) > 0
Submission	ck_submission_attempt_no	`AttemptNo` > 0
Submission	ck_submission_status	`SubmissionStatus` IN ('DRAFT', 'SUBMITTED')
Submission	ck_submission_score	`Score` IS NULL OR `Score` >= 0
Submission	ck_submission_grading	(`Score` IS NULL AND `GradedAt` IS NULL) OR (`Score` IS NOT NULL AND `GradedAt` IS NOT NULL)
SubmissionFile	ck_file_no	`FileNo` > 0
SubmissionFile	ck_file_size	`FileSize` >= 0
Quiz	ck_quiz_time	`OpenTime` < `CloseTime`
Quiz	ck_quiz_duration	`DurationMinutes` > 0
QuizAttempt	ck_quiz_attempt_no	`AttemptNo` > 0
QuizAttempt	ck_quiz_attempt_time	`FinishedAt` IS NULL OR `FinishedAt` >= `StartedAt`
QuizAttempt	ck_quiz_attempt_score	`Score` IS NULL OR `Score` >= 0
Grade	ck_grade_score	`Score` >= 0
Feedback	ck_feedback_content	CHAR_LENGTH(`Content`) > 0

Bảng 6: 15 chỉ mục hỗ trợ lọc và join

Index	Bảng	Cột
idx_user_account_status	UserAccount	`AccountStatus`
idx_user_last_access	UserAccount	`LastAccessAt`
idx_course_faculty	Course	`FacultyID`, `CourseCode`
idx_component_course_type	CourseComponent	`CourseID`, `ComponentType`
idx_section_schedule	CourseSection	`SemesterID`, `DayOfWeek`, `StartTime`, `EndTime`
idx_section_instructor_semester	CourseSection	`PrimaryInstructorID`, `SemesterID`
idx_enrollment_section_status	Enrollment	`SectionID`, `EnrollmentStatus`
idx_enrollment_student_status	Enrollment	`StudentID`, `EnrollmentStatus`
idx_announcement_section_posted	Announcement	`SectionID`, `PostedAt`
idx_resource_staff_type	LearningResource	`UploadedByStaffID`, `ResourceType`
idx_submission_assignment_status	Submission	`AssignmentID`, `SubmissionStatus`
idx_submission_student_submitted	Submission	`StudentID`, `SubmittedAt`

Index	Bảng	Cột
idx_quiz_attempt_student	QuizAttempt	`StudentID`, `StartedAt`
idx_grade_assessment_published	Grade	`AssessmentID`, `IsPublished`
idx_feedback_parent	Feedback	`ParentFeedbackID`

Bảng 7: Số hàng seed của 22 bảng (mọi bảng đạt ≥ 5)

#	Bảng	Số hàng	Đạt ≥ 5?
1	Admin	5	Có
2	Announcement	6	Có
3	Assessment	11	Có
4	Assignment	6	Có
5	AssignmentAttachment	6	Có
6	Course	5	Có
7	CourseComponent	8	Có
8	CourseSection	8	Có
9	Enrollment	16	Có
10	Faculty	5	Có
11	Feedback	7	Có
12	Grade	10	Có
13	LearningResource	8	Có
14	Quiz	5	Có
15	QuizAttempt	9	Có
16	Section_Resource	7	Có
17	Semester	5	Có
18	Staff	5	Có
19	Student	8	Có
20	Submission	13	Có
21	SubmissionFile	6	Có
22	UserAccount	18	Có

3.3 Các quyết định thiết kế quan trọng

3.3.1 ISA dùng chung khóa chính

`Student`, `Staff` và `Admin` đều có khóa chính `UserID` tham chiếu `UserAccount.UserID`. Cách này bảo đảm một tài khoản chỉ thuộc đúng một vai trò, tránh trùng lặp email/họ tên, và cho phép xóa cascade hồ sơ khi xóa tài khoản.

3.3.2 Bảng nối và thực thể yếu

`Section_Resource` là bảng nối N:M giữa lớp và tài nguyên, có khóa chính ghép `(SectionID, ResourceID)` và thuộc tính riêng `DisplayOrder`.
`AssignmentAttachment` và `SubmissionFile` là thực thể yếu: khóa chính của chúng chứa khóa của bảng cha, phù hợp với quan hệ 1:N.

3.3.3 Thuộc tính dẫn xuất

`Assignment.SubmissionCount` là thuộc tính dẫn xuất được lưu thật và được trigger cập nhật. Việc lưu giúp truy vấn thống kê nhanh, nhưng phải bảo đảm mọi thay đổi `Submission` đều kích hoạt tính lại. Đây là lý do có ba trigger `trg_submission_count_ai/ad/au`.

3.3.4 Phân tách ràng buộc

Ràng buộc miền giá trị dùng CHECK; ràng buộc duy nhất dùng UNIQUE; ràng buộc tham chiếu dùng FK; ràng buộc liên bản ghi dùng trigger; ràng buộc quy trình dùng procedure/service. Cách phân tách này tránh lạm dụng trigger và giữ schema dễ hiểu.

3.4 Quy tắc nghiệp vụ và nơi cài đặt

Bảng 8: Ánh xạ 17 quy tắc nghiệp vụ sang cơ chế cài đặt

RB	Nội dung tóm tắt	Cơ chế chính	Trạng thái
RB01	Tài khoản khóa/không hoạt động không được đăng nhập	Auth service kiểm tra `AccountStatus`	Đã cài
RB02	Mỗi lớp có tối đa một giảng viên phụ trách chính	Một cột `PrimaryInstructorID`	Đã cài
RB03	Không vượt sĩ số lớp	Trigger capacity + procedure	Đã cài
RB04	Môn có lý thuyết + thực hành phải đăng ký nhóm hợp lệ	Ứng dụng/seed; chưa ép bằng trigger	Một phần
RB05	Không trùng lịch giảng viên/phòng	Trigger schedule	Đã cài
RB06	Mỗi môn có đúng một lý thuyết, tối đa một thực hành	UNIQUE + seed/ứng dụng	Một phần
RB07	Chỉ Admin được khóa/mở tài khoản	`require_role("ADMIN")`	Đã cài
RB08	Chỉ Admin được tạo/sửa môn	`require_role("ADMIN")`	Đã cài

RB	Nội dung tóm tắt	Cơ chế chính	Trạng thái
RB09	Chỉ sinh viên ACTIVE trong lớp mới được nộp/làm quiz	Trigger submission + service	Đã cài
RB10	Bài nộp phải trong thời gian mở–hạn	Trigger submission	Đã cài
RB11	Không vượt số lần làm/submit	UNIQUE + trigger + service	Đã cài
RB12	Điểm nằm trong $[0, MaxScore]$	Trigger grade + CHECK	Đã cài
RB13	Chỉ giảng viên phụ trách được chấm	Staff service + trigger grade	Đã cài
RB14	Quiz có thời gian mở/đóng và thời lượng hợp lệ	CHECK quiz	Đã cài
RB15	Grade phải cùng section với assessment	Trigger grade	Đã cài
RB16	Feedback phải thuộc submission, parent cùng submission	Trigger feedback	Đã cài
RB17	Chỉ giảng viên phụ trách được tạo tài nguyên	Staff service	Đã cài

Ghi chú trung thực: RB04 và RB06 được kiểm soát chủ yếu ở dữ liệu mẫu và tầng ứng dụng, chưa có trigger ép buộc đầy đủ mọi tổ hợp đăng ký. Đây là giới hạn phạm vi của bản demo, không phải tuyên bố đã hoàn thiện mọi quy tắc của LMS thật.

4. Trigger

4.1 Vai trò và nguyên tắc

Trigger được dùng cho bốn nhóm việc mà CHECK/UNIQUE/FK không làm được:

1. **RB05 – trùng lịch**: phải so sánh khoảng thời gian với nhiều hàng khác trong cùng học kỳ.
2. **Thuộc tính dẫn xuất**: `Assignment.SubmissionCount` phải được tính lại sau mọi thay đổi `Submission`.
3. **Sĩ số**: phải đếm số enrollment ACTIVE trước khi thêm/đổi lớp.
4. **Kiểm tra liên bảng**: submission, grade và feedback cần JOIN nhiều bảng để xác định quyền và tính hợp lệ.

4.2 Danh sách 13 trigger

Bảng 9: 13 trigger và phạm vi bảo vệ

Trigger	Event / bảng	Mục đích
`trg_section_schedule_bi`	BEFORE INSERT `CourseSection`	Chặn trùng lịch giảng viên/phòng khi thêm lớp
`trg_section_schedule_bu`	BEFORE UPDATE `CourseSection`	Chặn trùng lịch khi sửa, bỏ qua chính lớp đang sửa
`trg_submission_count_ai`	AFTER INSERT `Submission`	Tăng/tính lại `SubmissionCount`
`trg_submission_count_ad`	AFTER DELETE `Submission`	Giảm/tính lại `SubmissionCount`
`trg_submission_count_au`	AFTER UPDATE `Submission`	Tính lại assignment cũ và mới nếu đổi assignment
`trg_enrollment_capacity_bi`	BEFORE INSERT `Enrollment`	Chặn vượt sĩ số khi ghi danh
`trg_enrollment_capacity_bu`	BEFORE UPDATE `Enrollment`	Chặn vượt sĩ số khi đổi lớp/trạng thái
`trg_submission_validate_bi`	BEFORE INSERT `Submission`	Kiểm tra assignment, ghi danh ACTIVE, thời gian, số lần, điểm, giảng viên
`trg_submission_validate_bu`	BEFORE UPDATE `Submission`	Kiểm tra lại các quy tắc trên khi sửa
`trg_grade_validate_bi`	BEFORE INSERT `Grade`	Kiểm tra enrollment/assessment cùng section, điểm, giảng viên
`trg_grade_validate_bu`	BEFORE UPDATE `Grade`	Kiểm tra lại khi sửa điểm/assessment
`trg_feedback_validate_bi`	BEFORE INSERT `Feedback`	Feedback thuộc submission, parent cùng submission, người viết hợp lệ
`trg_feedback_validate_bu`	BEFORE UPDATE `Feedback`	Kiểm tra lại khi sửa feedback

4.3 Trigger trùng lịch RB05

Điều kiện giao khoảng của hai lớp trong cùng `SemesterID` và cùng `DayOfWeek` là:

$$\texttt{NEW.StartTime} < \texttt{cs.EndTime} \quad \wedge \quad \texttt{cs.StartTime} < \texttt{NEW.EndTime}.$$

Trigger kiểm tra cả `PrimaryInstructorID` và `RoomCode`; khi UPDATE phải loại trừ chính hàng đang sửa bằng `cs.SectionID <> OLD.SectionID`. Test âm `04_trigger_expected_fail_schedule.sql` và `11_trigger_expected_fail_schedule_update.sql` chứng minh cả INSERT và UPDATE đều bị chặn.

4.4 Trigger thuộc tính dẫn xuất

Công thức:

$$\texttt{SubmissionCount} = \left|\{\, s \in \texttt{Submission} \mid s.\texttt{AssignmentID} = a.\texttt{AssessmentID} \,\}\right|.$$

Ba trigger AFTER INSERT/DELETE/UPDATE luôn đếm lại từ bảng thật, không cộng trừ mù. Khi UPDATE đổi `AssignmentID`, trigger tính lại cả assignment cũ và mới. Test `10_trigger_dml_matrix.sql` kiểm tra insert nhiều dòng, update đổi assignment, update cột không khóa, delete và rollback.

4.5 Ma trận DML

Bảng 10: Ma trận DML được bảo vệ bởi trigger

Nhóm	Bảng	DML
Lịch học	`CourseSection`	INSERT, UPDATE
Sĩ số	`Enrollment`	INSERT, UPDATE
Bài nộp	`Submission`	INSERT, UPDATE, DELETE
Điểm	`Grade`	INSERT, UPDATE
Phản hồi	`Feedback`	INSERT, UPDATE

4.6 Mã tiêu biểu

Listing 1: Trigger chặn trùng lịch khi INSERT (trích `05_triggers.sql`)

```
CREATE TRIGGER `trg_section_schedule_bi`
BEFORE INSERT ON `CourseSection`
FOR EACH ROW
BEGIN
  IF EXISTS (
    SELECT 1
      FROM `CourseSection` cs
     WHERE cs.`SemesterID` = NEW.`SemesterID`
       AND cs.`DayOfWeek` = NEW.`DayOfWeek`
       AND NEW.`StartTime` < cs.`EndTime`
       AND cs.`StartTime` < NEW.`EndTime`
       AND (cs.`PrimaryInstructorID` = NEW.`PrimaryInstructorID`
            OR cs.`RoomCode` = NEW.`RoomCode`)
  ) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Giang vien hoac phong bi trung lich trong hoc ky.';
```

```
17    END IF;
18  END
```

5. Stored procedure

5.1 Danh sách procedure

Bảng 11: 5 stored procedure và mục đích

Procedure	Tham số	Mục đích
`sp_InsertEnrollment`	`student_id`, `section_id`, `status`	Thêm ghi danh có kiểm tra tài khoản, lớp, trùng, sĩ số, transaction
`sp_UpdateEnrollment`	`enrollment_id`, `section_id`, `status`	Sửa lớp/trạng thái; không đổi lớp/rút nếu đã có điểm
`sp_DeleteEnrollment`	`enrollment_id`	Xóa ghi danh nếu chưa có điểm
`sp_GetStudentEnrollments`	`student_id`, `semester_id`, `status`, `limit`, `offset`	Liệt kê ghi danh có JOIN, WHERE, ORDER BY, phân trang
`sp_GetAssignmentStatistics`	`section_id`, `semester_id`, `min_submissions`	Thống kê bài tập có aggregate, GROUP BY, HAVING

5.2 CRUD trên `Enrollment`

Ba procedure CRUD thực hiện các kiểm tra sau:

- `sp_InsertEnrollment`: StudentID/SectionID dương; student tồn tại và tài khoản ACTIVE; section tồn tại; section OPEN; không trùng `(StudentID, SectionID)`; nếu status ACTIVE thì đếm sĩ số so với `Capacity`; khóa section bằng `FOR UPDATE` để tránh đua.
- `sp_UpdateEnrollment`: enrollment tồn tại; status hợp lệ; section mới tồn tại/OPEN; không trùng; kiểm tra sĩ số loại trừ chính enrollment đang sửa; không cho đổi lớp hoặc rút môn nếu đã có `Grade`.
- `sp_DeleteEnrollment`: enrollment tồn tại; từ chối xóa nếu có `Grade`; xóa cứng khi chưa có điểm.

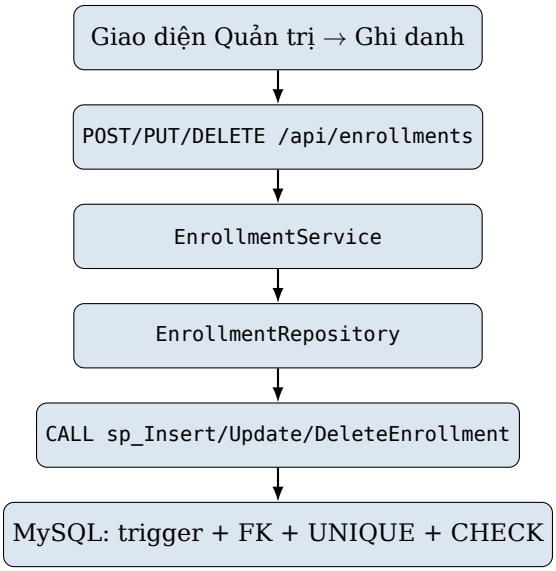
Cả ba dùng `START TRANSACTION`, `COMMIT` và handler `EXIT HANDLER FOR SQLEXCEPTION` để `ROLLBACK` rồi `RESIGNAL`. Nhờ đó lỗi nghiệp vụ trả về mã `45000` và được API ánh xạ thành HTTP 400 với mã `database_rule_violation`.

5.3 Procedure truy vấn

Bảng 12: Đối chiếu yêu cầu của hai procedure truy vấn

Yêu cầu	`sp_GetStudentEnrollments`	`sp_GetAssignmentStatistics`
JOIN ≥ 2 bảng	7 bảng	8 bảng
WHERE	student, semester, status	section, semester
ORDER BY	năm, học kỳ, mã môn, mã lớp	tỷ lệ hoàn thành, mã môn, mã lớp
Aggregate	Không	COUNT, MAX
GROUP BY / HAVING	Không	Có, theo ngưỡng bài nộp
Tham số UI	tab Ghi danh	tab Thống kê

5.4 Luồng UI – API – procedure



Hình 3: Đường đi của một thao tác CRUD Ghi danh

5.5 Mã tiêu biểu

Listing 2: Phần kiểm tra và transaction của `sp_InsertEnrollment`

```
START TRANSACTION;

SELECT COUNT(*) INTO v_exists
  FROM `Student` s
  JOIN `UserAccount` u ON u.`UserID` = s.`UserID`
 WHERE s.`UserID` = p_student_id
   AND u.`AccountStatus` = 'ACTIVE';
```

```
8
9  IF v_exists = 0 THEN
10   SIGNAL SQLSTATE '45000'
11     SET MESSAGE_TEXT = 'Sinh vien khong ton tai hoac tai khoan khong hoat dong.';
12 END IF;
13
14 SELECT `Capacity`, `SectionStatus`
15   INTO v_capacity, v_section_status
16   FROM `CourseSection`
17  WHERE `SectionID` = p_section_id
18  FOR UPDATE;
19
20 IF v_section_status <> 'OPEN' THEN
21   SIGNAL SQLSTATE '45000'
22     SET MESSAGE_TEXT = 'Lop hoc phan khong o trang thai OPEN.';
23 END IF;
24
25 INSERT INTO `Enrollment`
26   (`StudentID`, `SectionID`, `EnrollmentDate`, `EnrollmentStatus`)
27 VALUES
28   (p_student_id, p_section_id, CURRENT_TIMESTAMP, v_status);
29
30 SET v_new_id = LAST_INSERT_ID();
31 COMMIT;
```

6. Stored function

6.1 Yêu cầu và cách đáp ứng

Hai function đều `READS SQL DATA`, không ghi dữ liệu, có cursor, handler `NOT FOUND`, vòng lặp `LOOP`, `IF`, validate đầu vào và truy vấn dữ liệu.

Bảng 13: Đối chiếu yêu cầu rubric của hai function

Yêu cầu	Bằng chứng
Cursor	`DECLARE cur_enrolled CURSOR FOR ...; DECLARE cur_grades CURSOR FOR ...`
Handler	`DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE`
OPEN/FETCH/LOOP/CLOSE	Có đầy đủ trong cả hai function
IF/LOOP tính toán	Đếm submission; cộng điểm phần trăm đã publish
Query dữ liệu	JOIN `Assignment/Assessment/Enrollment/Submission`; JOIN `Grade/Assessment`
Validate input	ID NULL hoặc ≤ 0 thì `SIGNAL`; kiểm tra bản ghi tồn tại
Kết quả	`DECIMAL(5,2)`, làm tròn 2 chữ số; trả 0.00 khi không có dữ liệu

6.2 `fn_CalculateAssignmentCompletionRate`

Công thức: số sinh viên ACTIVE đã có ít nhất một submission trạng thái `SUBMITTED` chia cho tổng số sinh viên ACTIVE của lớp, nhân 100. Nếu lớp không có sinh viên ACTIVE, hàm trả `0.00`. Hàm kiểm tra `AssignmentID` dương và assignment tồn tại trước khi mở cursor.

6.3 `fn_CalculateStudentAverageScore`

Hàm duyệt các `Grade` đã publish của một enrollment, quy đổi mỗi điểm theo `Score * 100 / MaxScore`, cộng dồn và chia cho số assessment đã publish. Đây là chỉ số phân tích nội bộ, không phải công thức điểm chính thức của HCMUT. Nếu chưa có grade published, hàm trả `0.00`.

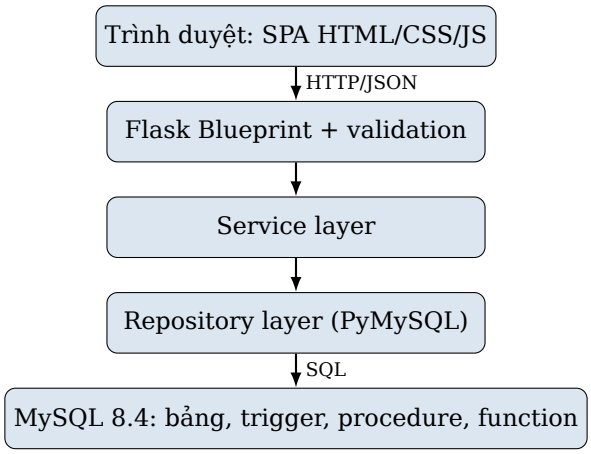
6.4 Kết quả kiểm thử

Bảng 14: Kết quả gọi hàm trên dữ liệu mẫu

Biểu thức	Kết quả
`fn_CalculateAssignmentCompletionRate(1)`	100.00
`fn_CalculateAssignmentCompletionRate(6)`	0.00
`fn_CalculateAssignmentCompletionRate(0)`	SIGNAL (expected fail)
`fn_CalculateStudentAverageScore(1)`	82.50
`fn_CalculateStudentAverageScore(8)`	0.00
`fn_CalculateStudentAverageScore(-1)`	SIGNAL (expected fail)

7. Ứng dụng

7.1 Kiến trúc



Hình 4: Kiến trúc bốn lớp của ứng dụng

7.2 Phân quyền theo vai trò

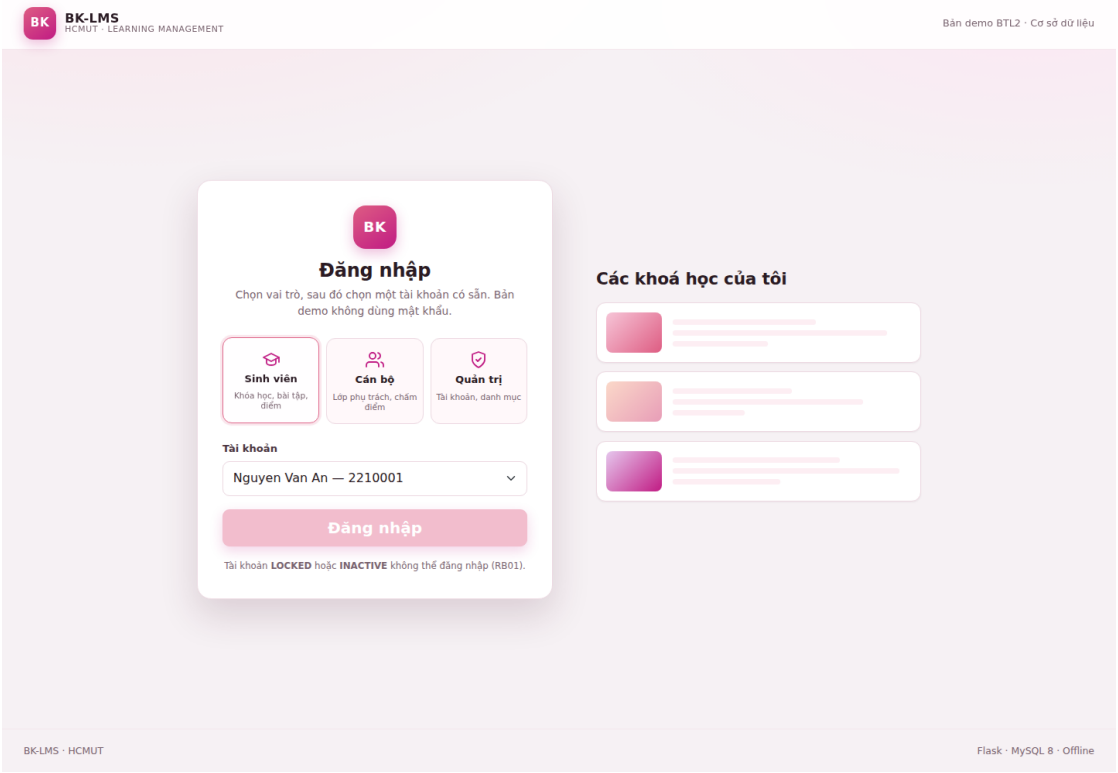
Ứng dụng dùng mô hình demo chọn vai trò rồi chọn tài khoản, không có mật khẩu. Sau khi đăng nhập, Flask ghi `user_id` và `role` vào session. Mọi API được bảo vệ bằng `@require_role(...)`; thiếu session trả 401, sai vai trò trả 403. Ngoài vai trò, service còn kiểm tra quyền sở hữu dữ liệu: Student chỉ thấy dữ liệu của mình, Staff chỉ thấy lớp mình phụ trách, Admin không được chấm điểm (RB13).

Bảng 15: Nhóm API và vai trò yêu cầu

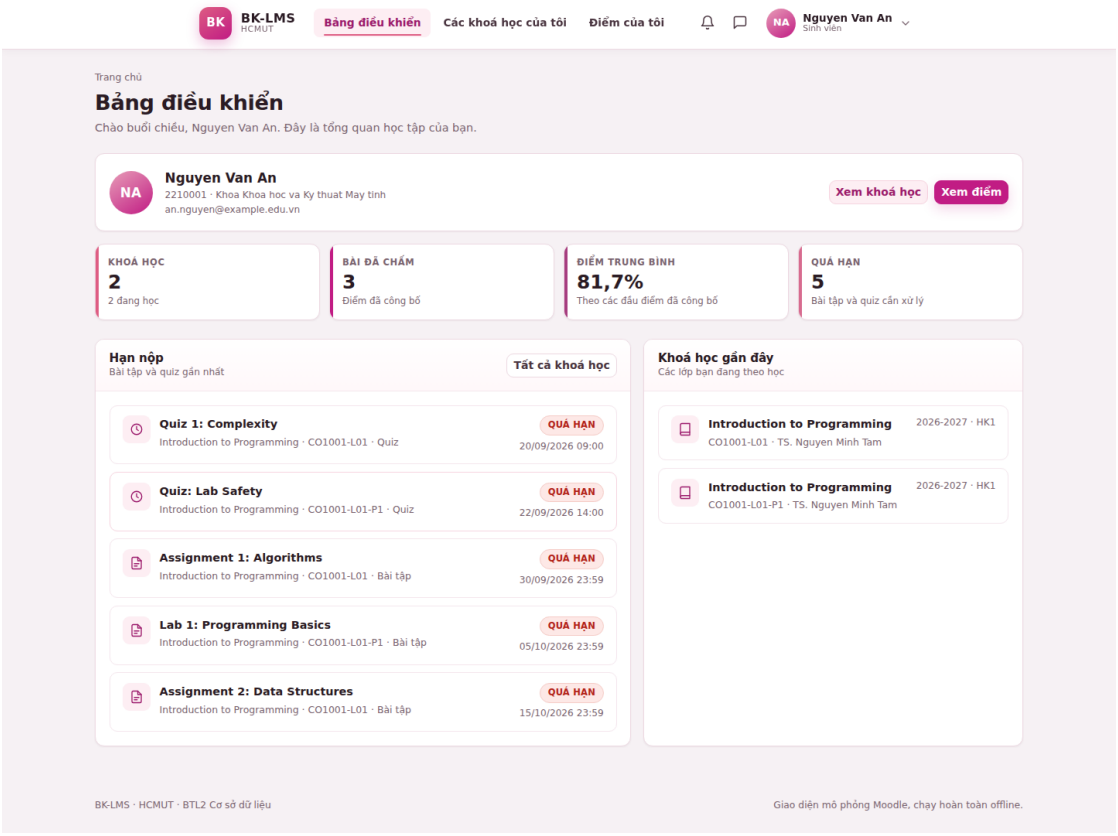
Nhóm	Endpoint tiêu biểu	Vai trò
Health	`/api/health`, `/api/health/live`	Public
Auth	`/api/auth/roles`, `/api/auth/accounts`, `/api/auth/login`	Public (demo)
Auth	`/api/auth/me`, `/api/auth/logout`	Session
Catalog	`/api/meta`, `/api/semesters`, `/api/sections`, `/api/courses`	Public
Catalog	`/api/students`	ADMIN
Enrollment	`/api/enrollments`, `/api/enrollments/<id>`	ADMIN
Statistics	`/api/statistics/overview`, `.../assignments`	ADMIN
Statistics	`.../completion`, `.../average`	ADMIN
Student	`/api/student/*`	STUDENT
Staff	`/api/staff/*`	STAFF
Admin	`/api/admin/*`	ADMIN

7.3 Giao diện

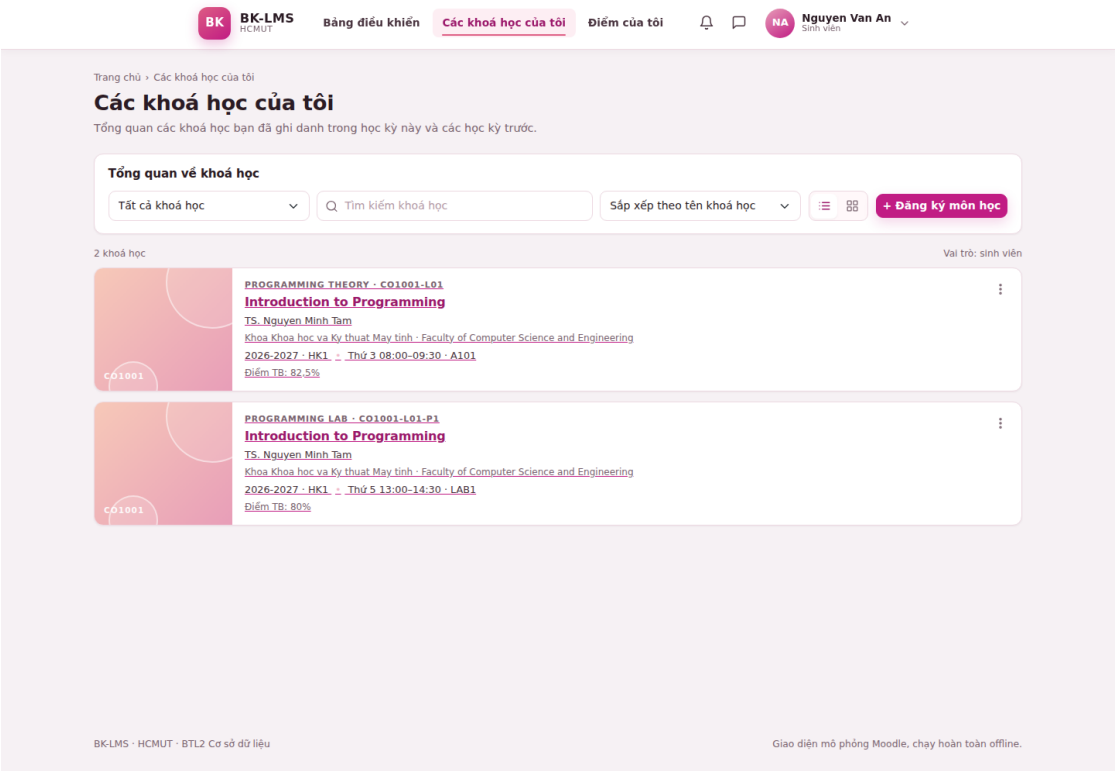
Giao diện dùng Bootstrap đã vendored, không gọi CDN, nên chạy được offline. SPA dùng hash router với các trang `#/dashboard`, `#/courses`, `#/course/<id>`, `#/grades`, `#/profile`, `#/admin?tab=...`. Các modal thêm/sửa/xóa dùng Bootstrap modal thay vì `window.prompt()`.



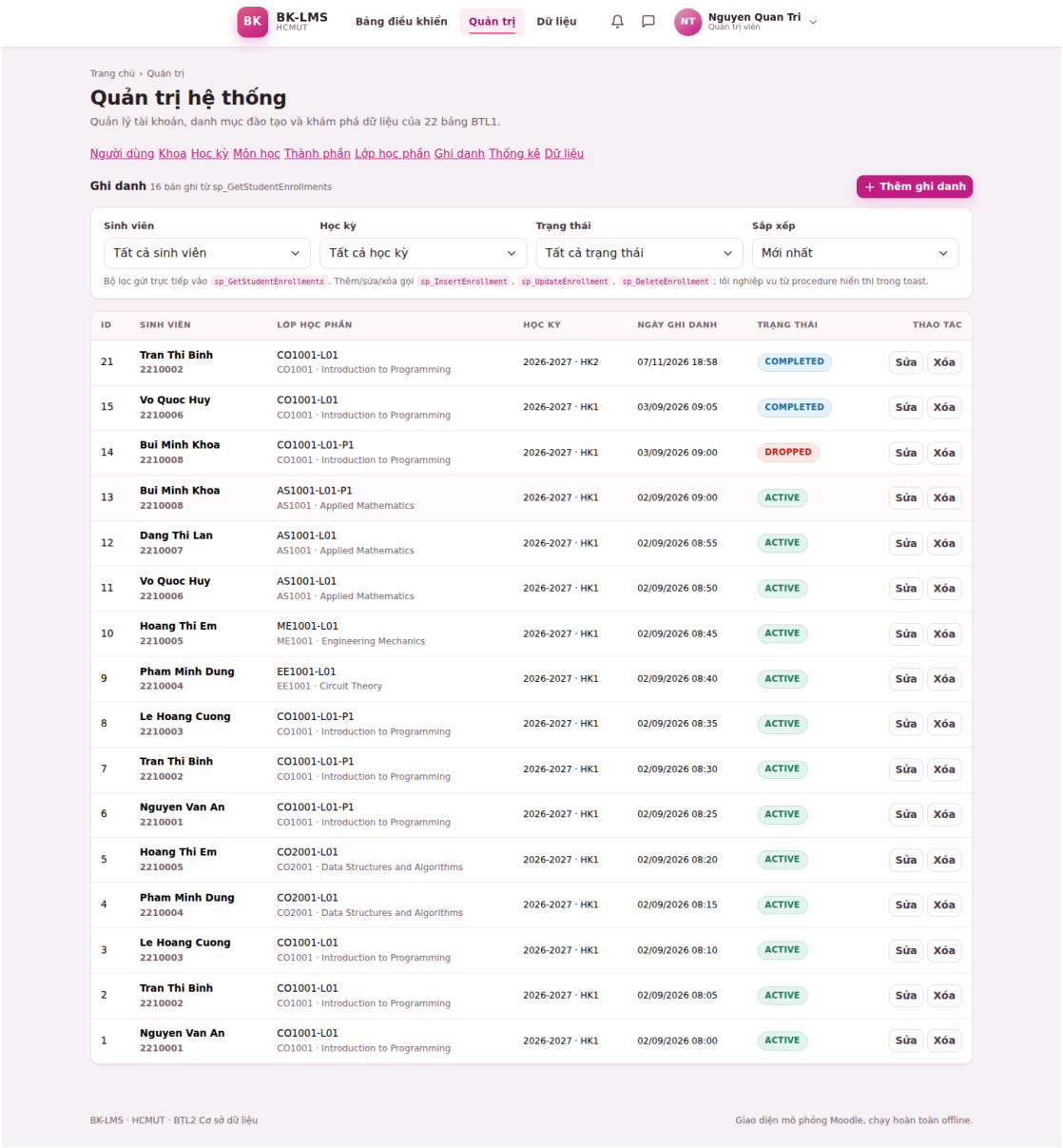
Hình 5: Màn hình chọn vai trò và tài khoản demo



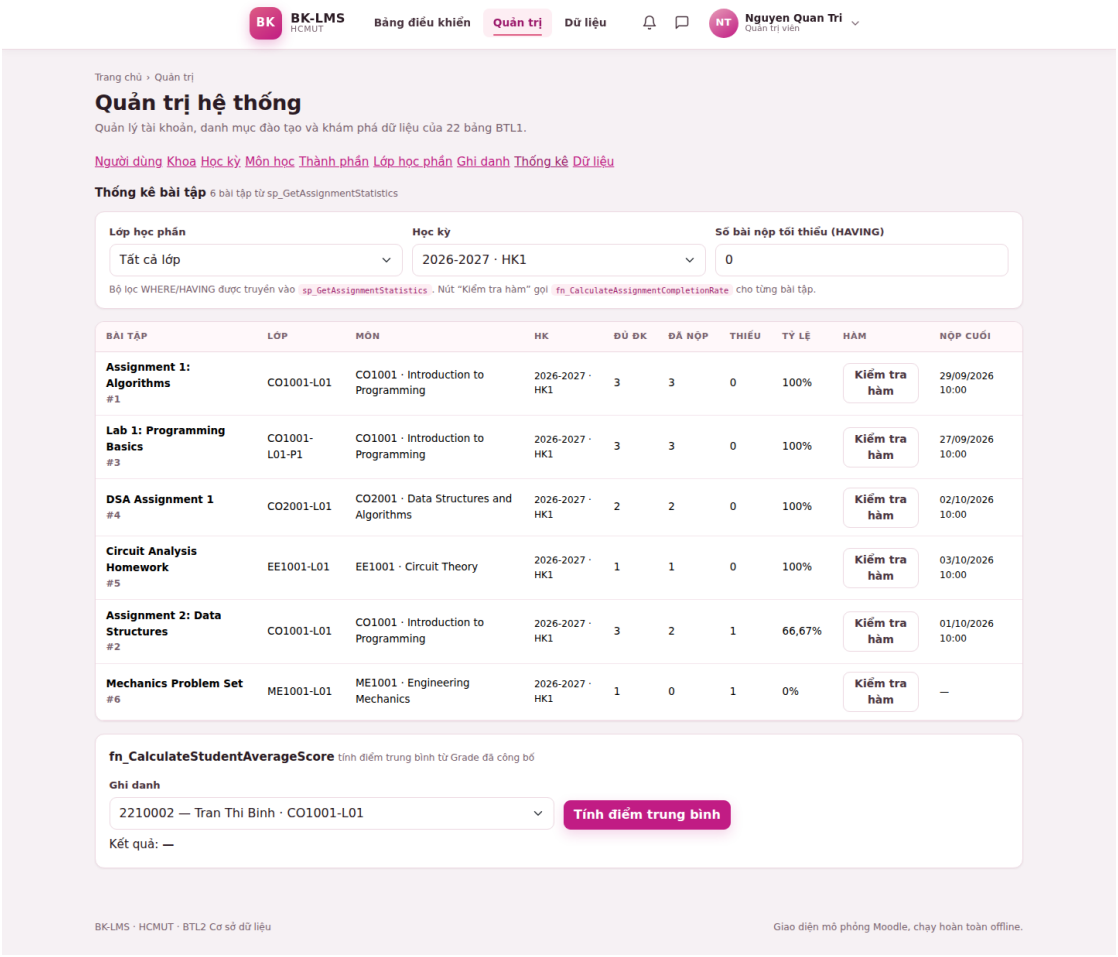
Hình 6: Dashboard sinh viên



Hình 7: Danh sách lớp học phần của sinh viên



Hình 8: Tab Quản trị → Ghi danh với bộ lọc và CRUD



Hình 9: Tab Quản trị → Thống kê gọi procedure và function

7.4 Luồng CRUD Ghi danh

Bảng 16 tóm tắt đường đi của từng thao tác. Danh sách lấy dữ liệu từ `sp_GetStudentEnrollments`; thêm/sửa/xóa gọi đúng ba procedure CRUD. Lỗi nghiệp vụ từ procedure được hiển thị bằng toast đỏ, không làm mất dữ liệu đang nhập.

Bảng 16: Luồng UI – API – procedure của CRUD Ghi danh

Thao tác	HTTP	Service/Repository	Procedure
Danh sách	`GET /api/enrollments`	`list`	`sp_GetStudentEnrollments`
Thêm	`POST /api/enrollments`	`create`	`sp_InsertEnrollment`
Sửa	`PUT /api/enrollments/<id>`	`update`	`sp_UpdateEnrollment`
Xóa	`DELETE /api/enrollments/<id>`	`delete`	`sp_DeleteEnrollment`

7.5 Cấu hình và chạy

Ứng dụng chạy bằng `uv` và Python 3.12; CSDL chạy bằng Docker MySQL 8.4. Các lệnh chính:

Listing 3: Lệnh chạy ứng dụng và kiểm thử

```
cd hcmut-lms-btl2
uv sync
cp .env.example .env
docker compose up -d mysql
.venv/bin/python scripts/init_database.py
.venv/bin/python run.py
# terminal khac
uv run pytest -q
.venv/bin/python scripts/verify_database.py
.venv/bin/python scripts/smoke_api.py
```

8. Kiểm thử

8.1 Chiến lược kiểm thử

Bảng 17: Các tầng kiểm thử và phạm vi

Tầng	Nội dung	Kết quả
CSDL	22 bảng, 13 trigger, 5 procedure, 2 function, seed, ràng buộc	PASS
SQL test	13 file test dương/âm, expected fail	PASS
Python	API, service, repository, validation, auth, frontend static	72 passed
Smoke API	Flask + MySQL thật, 3 vai trò, CRUD, 401/403	31/31 PASS
Browser E2E	Chromium headless: Student, Staff, Admin, CRUD Ghi danh	PASS, không lỗi console

8.2 Kiểm thử CSDL

Script `scripts/verify_database.py` kiểm tra số bảng, trigger, procedure, function, tính nhất quán của `SubmissionCount`, xung đột lịch, kết quả hai function và số hàng seed tối thiểu. Kết quả thật:

Listing 4: Kết quả kiểm tra CSDL

```
[PASS] table_count = 22
[PASS] trigger_count = 13
[PASS] procedure_count = 5
[PASS] function_count = 2
```

```
[PASS] submission_count_mismatches = 0
[PASS] schedule_conflicts = 0
[PASS] completion_rate_assignment_1 = 100.00
[PASS] completion_rate_assignment_6 = 0.00
[PASS] average_score_enrollment_1 = 82.50
[PASS] every one of the 22 tables has at least 5 seed rows
DATABASE VERIFICATION: PASS
```

8.3 Kiểm thử Python

Bảng 18: Các nhóm test Python

File	Nội dung
`test_auth.py`	login/logout, role guard, profile, catalog, enroll/drop, quiz, file, không PII
`test_config.py`	fail-fast SECRET_KEY ở production
`test_enrollments_api.py`	CRUD, validation, lỗi procedure, 401/403/200
`test_statistics_api.py`	thống kê, function, 401/403/200
`test_catalog_api.py`	catalog, bảo vệ `/api/students`
`test_repositories.py`	xác nhận gọi đúng `sp_*` / `fn_*`
`test_health.py`, `test_validation.py`	health, validation
`test_json_provider.py`, `test_frontend.py`	JSON provider, frontend static

Kết quả: **72 passed**. Các test 401/403 chứng minh API không chỉ ẩn UI mà thực sự chặn truy cập trực tiếp.

8.4 Smoke API và phân quyền

Smoke chạy trên Flask + MySQL thật, gồm 31 check. Bảng 19 là các kiểm tra phân quyền trọng tâm.

Bảng 19: Kiểm tra 401/403/200 trên API quản trị

Request	Không login	Student	Admin
`GET /api/enrollments`	401	403	200
`POST /api/enrollments`	401	403	201
`GET /api/statistics/overview`	401	403	200
`GET /api/students`	401	403	200

8.5 Kiểm thử trình duyệt

Playwright Chromium headless đã chạy các luồng: đăng nhập ba vai trò; Student xem dashboard, courses, course tabs, assignment, quiz, grades, profile; Staff xem dashboard, submissions, chấm điểm; Admin xem users/sections, CRUD Ghi danh, Thống kê + hai function, data explorer. Kết quả không có `pageerror` hay lỗi console; ảnh được lưu trong `docs/evidence/`. Chuỗi CRUD thực tế: bảng có 16 dòng, lọc student 101 còn 2 dòng, thêm một dòng, sửa sang `DROPPED`, xóa và trở về 16 dòng.

8.6 Readiness và việc còn lại

Script `scripts/check_submission_readiness.py` kiểm tra tài liệu, số test, cấu trúc source và các artefact bắt buộc. Kết quả hiện tại chỉ còn FAIL vì `docs/TEAM_ASSIGNMENT.md` chưa điền tên/MSSV. Đây là P0 của con người, không phải lỗi code.

9. Phân công, kết luận và hạn chế

9.1 Phân công thành viên

Bảng 20: Phân công 5 thành viên và object sở hữu

TV	Họ tên – MSSV	Vai trò	SQL object / phần phụ trách
TV1	[Họ tên] – [MSSV]	Database lead	`trg_section_schedule_bi/bu`, `trg_feedback_validate_bi/bu`
TV2	[Họ tên] – [MSSV]	CRUD & backend	`sp_Insert/Update/DeleteEnrollment`, API/UI Ghi danh
TV3	[Họ tên] – [MSSV]	Integrity & seed	`trg_enrollment_capacity_bi/bu`, `trg_grade_validate_bi/bu`
TV4	[Họ tên] – [MSSV]	Query & function	2 query procedure, 2 function, tab Thống kê
TV5	[Họ tên] – [MSSV]	Submission & UI/ops	5 trigger submission, frontend, script Windows

Mỗi thành viên phải điền tên/MSSV, commit gắn với object mình sở hữu và chạy `git shortlog -sne --all` để đối chiếu. Bảng đầy đủ và câu hỏi demo nằm trong `docs/TEAM_ASSIGNMENT.md` và `docs/DEMO_SCRIPT.md`.

9.2 Kết luận

Bài tập đã hiện thực đầy đủ mô hình BTL1 thành 22 bảng quan hệ với 35 khóa ngoại, 10 UNIQUE, 39 CHECK và 15 index. Các quy tắc liên bảng được cài đặt bằng 13 trigger; nghiệp vụ ghi danh được đóng gói trong 3 procedure CRUD có transaction; hai procedure truy vấn đáp ứng JOIN/WHERE/ORDER BY/aggregate/GROUP BY/HAVING; hai function có cursor/IF/LOOP và được gọi thật từ giao diện. Ứng dụng Flask + SPA phân quyền ba vai trò, gọi procedure/function thật, và đã được kiểm thử ở mức CSDL, API và trình duyệt.

9.3 Hạn chế và hướng phát triển

Bảng 21: Hạn chế đã biết và hướng khắc phục

Hạn chế	Hướng khắc phục
Không có xác thực bằng mật khẩu; chỉ chọn tài khoản demo	Tích hợp CAS/OAuth hoặc password hash + session thật
Chưa có CSRF token	Thêm CSRF token cho form/POST hoặc dùng SameSite + token
Chưa kiểm thử Windows thật	Chạy `setup.bat` trên máy Windows sạch, ghi log
Chưa kiểm thử Firefox/Safari/Edge	Bổ sung Playwright đa trình duyệt
Chưa kiểm thử tải/đồng thời quy mô lớn	Dùng Locust/k6, đo khóa `FOR UPDATE` và deadlock
RB04/RB06 chưa ép đầy đủ bằng trigger	Bổ sung kiểm tra nhóm lý thuyết–thực hành ở procedure/service
Tệp đính kèm chỉ lưu metadata	Bổ sung storage thật, kiểm tra MIME/size, virus scan

A. Mã schema tiêu biểu

Toàn bộ DDL nằm trong `database/schema/01_tables.sql`. Dưới đây là phần đầu của file, gồm các bảng định danh và học vụ.

Listing 5: Trích `01_tables.sql`: bảng định danh và học vụ

```sql
USE `hcmut_lms`;

-- Tables are created without foreign keys so the key phase is explicit.
-- MySQL 8.0+, InnoDB, utf8mb4.

CREATE TABLE IF NOT EXISTS `UserAccount` (
  `UserID` INT NOT NULL AUTO_INCREMENT,
  `Email` VARCHAR(255) NOT NULL,
  `FullName` VARCHAR(150) NOT NULL,
  `AvatarURL` VARCHAR(500) NOT NULL,
  `AccountStatus` VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  `LastAccessAt` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`UserID`),
  UNIQUE KEY `uq_useraccount_email` (`Email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Faculty` (
  `FacultyID` INT NOT NULL AUTO_INCREMENT,
  `FacultyName` VARCHAR(150) NOT NULL,
  `FacultyNameEN` VARCHAR(150) NULL,
  PRIMARY KEY (`FacultyID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Student` (
  `UserID` INT NOT NULL,
  `StudentCode` VARCHAR(20) NOT NULL,
  `FacultyID` INT NOT NULL,
  PRIMARY KEY (`UserID`),
  UNIQUE KEY `uq_student_code` (`StudentCode`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Staff` (
  `UserID` INT NOT NULL,
  `FacultyID` INT NOT NULL,
  PRIMARY KEY (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Admin` (
  `UserID` INT NOT NULL,
  PRIMARY KEY (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Semester` (
  `SemesterID` INT NOT NULL AUTO_INCREMENT,
```

```
  `AcademicYear` VARCHAR(9) NOT NULL,
  `TermNo` TINYINT NOT NULL,
  `StartDate` DATE NOT NULL,
  `EndDate` DATE NOT NULL,
  PRIMARY KEY (`SemesterID`),
  UNIQUE KEY `uq_semester_year_term` (`AcademicYear`, `TermNo`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Course` (
  `CourseID` INT NOT NULL AUTO_INCREMENT,
  `CourseCode` VARCHAR(20) NOT NULL,
  `CourseName` VARCHAR(200) NOT NULL,
  `Credits` TINYINT NOT NULL,
  `FacultyID` INT NOT NULL,
  `CreatedByAdminID` INT NOT NULL,
  PRIMARY KEY (`CourseID`),
  UNIQUE KEY `uq_course_code` (`CourseCode`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `CourseComponent` (
  `ComponentID` INT NOT NULL AUTO_INCREMENT,
  `CourseID` INT NOT NULL,
  `ComponentType` VARCHAR(20) NOT NULL,
  `ComponentName` VARCHAR(150) NOT NULL,
  PRIMARY KEY (`ComponentID`),
  UNIQUE KEY `uq_course_component_type` (`CourseID`, `ComponentType`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `CourseSection` (
  `SectionID` INT NOT NULL AUTO_INCREMENT,
  `ComponentID` INT NOT NULL,
  `SemesterID` INT NOT NULL,
  `PrimaryInstructorID` INT NULL,
  `SectionCode` VARCHAR(30) NOT NULL,
  `Capacity` SMALLINT NOT NULL,
  `SectionStatus` VARCHAR(20) NOT NULL DEFAULT 'OPEN',
  `DayOfWeek` TINYINT NOT NULL,
  `StartTime` TIME NOT NULL,
  `EndTime` TIME NOT NULL,
  `RoomCode` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`SectionID`),
  UNIQUE KEY `uq_section_semester_code` (`SemesterID`, `SectionCode`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Enrollment` (
  `EnrollmentID` INT NOT NULL AUTO_INCREMENT,
  `StudentID` INT NOT NULL,
  `SectionID` INT NOT NULL,
  `EnrollmentDate` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `EnrollmentStatus` VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (`EnrollmentID`),
```

```
  UNIQUE KEY `uq_enrollment_student_section` (`StudentID`, `SectionID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `Announcement` (
  `AnnouncementID` INT NOT NULL AUTO_INCREMENT,
```

B. Mã trigger tiêu biểu

Listing 6: Trigger `trg_section_schedule_bi`

```
CREATE TRIGGER `trg_section_schedule_bi`
BEFORE INSERT ON `CourseSection`
FOR EACH ROW
BEGIN
  DECLARE v_conflict INT DEFAULT 0;

  IF NEW.`PrimaryInstructorID` IS NOT NULL THEN
    SELECT COUNT(*) INTO v_conflict
    FROM `CourseSection` cs
    WHERE cs.`SemesterID` = NEW.`SemesterID`
      AND cs.`PrimaryInstructorID` = NEW.`PrimaryInstructorID`
      AND cs.`DayOfWeek` = NEW.`DayOfWeek`
      AND NEW.`StartTime` < cs.`EndTime`
      AND cs.`StartTime` < NEW.`EndTime`;

    IF v_conflict > 0 THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Trung lich giang vien trong cung hoc ky.';
    END IF;
  END IF;

  SELECT COUNT(*) INTO v_conflict
  FROM `CourseSection` cs
  WHERE cs.`SemesterID` = NEW.`SemesterID`
    AND cs.`RoomCode` = NEW.`RoomCode`
    AND cs.`DayOfWeek` = NEW.`DayOfWeek`
    AND NEW.`StartTime` < cs.`EndTime`
    AND cs.`StartTime` < NEW.`EndTime`;

  IF v_conflict > 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Trung phong hoc trong cung hoc ky.';
  END IF;
END$$
```

Listing 7: Trigger `trg_submission_count_au`

```
CREATE TRIGGER `trg_submission_count_au`
AFTER UPDATE ON `Submission`
FOR EACH ROW
```

```
BEGIN
  UPDATE `Assignment`
     SET `SubmissionCount` = (
       SELECT COUNT(*) FROM `Submission`
        WHERE `AssignmentID` = OLD.`AssignmentID`
     )
   WHERE `AssessmentID` = OLD.`AssignmentID`;

  IF NEW.`AssignmentID` <> OLD.`AssignmentID` THEN
    UPDATE `Assignment`
       SET `SubmissionCount` = (
         SELECT COUNT(*) FROM `Submission`
          WHERE `AssignmentID` = NEW.`AssignmentID`
       )
     WHERE `AssessmentID` = NEW.`AssignmentID`;
  END IF;
END$$

-- ======================================================================
-- Additional integrity triggers for cross-row rules that CHECK cannot
-- express. They are part of the database foundation, not replacements
-- for the two required BTL2 triggers above.
```

C. Mã procedure tiêu biểu

Listing 8: `sp_InsertEnrollment`: phần đầu và validation

```
CREATE PROCEDURE `sp_InsertEnrollment`(
  IN p_student_id INT,
  IN p_section_id INT,
  IN p_enrollment_status VARCHAR(20)
)
BEGIN
  DECLARE v_exists INT DEFAULT 0;
  DECLARE v_status VARCHAR(20);
  DECLARE v_section_status VARCHAR(20);
  DECLARE v_capacity INT DEFAULT 0;
  DECLARE v_active_count INT DEFAULT 0;
  DECLARE v_new_id INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  IF p_student_id IS NULL OR p_student_id <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'StudentID phai la so nguyen duong.';
  END IF;
```

```
24
25  IF p_section_id IS NULL OR p_section_id <= 0 THEN
26    SIGNAL SQLSTATE '45000'
27      SET MESSAGE_TEXT = 'SectionID phai la so nguyen duong.';
28  END IF;
29
30  SET v_status = COALESCE(p_enrollment_status, 'ACTIVE');
31  IF v_status NOT IN ('ACTIVE', 'DROPPED', 'COMPLETED') THEN
32    SIGNAL SQLSTATE '45000'
33      SET MESSAGE_TEXT = 'EnrollmentStatus chi nhan ACTIVE, DROPPED hoac COMPLETED.';
34  END IF;
35
36  START TRANSACTION;
37
38  SELECT COUNT(*) INTO v_exists
39    FROM `Student` s
40    JOIN `UserAccount` u ON u.`UserID` = s.`UserID`
41   WHERE s.`UserID` = p_student_id
42     AND u.`AccountStatus` = 'ACTIVE';
43
44  IF v_exists = 0 THEN
45    SIGNAL SQLSTATE '45000'
```

Listing 9: `sp_InsertEnrollment`: transaction và insert

```
1      SET MESSAGE_TEXT = 'Sinh vien khong ton tai hoac tai khoan khong hoat dong.';
2  END IF;
3
4  SELECT COUNT(*) INTO v_exists
5    FROM `CourseSection`
6   WHERE `SectionID` = p_section_id;
7
8  IF v_exists = 0 THEN
9    SIGNAL SQLSTATE '45000'
10     SET MESSAGE_TEXT = 'Lop hoc phan khong ton tai.';
11 END IF;
12
13 SELECT `Capacity`, `SectionStatus`
14   INTO v_capacity, v_section_status
15   FROM `CourseSection`
16  WHERE `SectionID` = p_section_id
17  FOR UPDATE;
18
19 IF v_section_status <> 'OPEN' THEN
20   SIGNAL SQLSTATE '45000'
21     SET MESSAGE_TEXT = 'Lop hoc phan khong o trang thai OPEN.';
22 END IF;
23
24 SELECT COUNT(*) INTO v_exists
25   FROM `Enrollment`
26  WHERE `StudentID` = p_student_id
27    AND `SectionID` = p_section_id;
```

```
28
29    IF v_exists > 0 THEN
30      SIGNAL SQLSTATE '45000'
31        SET MESSAGE_TEXT = 'Sinh vien da co ghi danh trong lop nay.';
32    END IF;
33
34    IF v_status = 'ACTIVE' THEN
35      SELECT COUNT(*) INTO v_active_count
36        FROM `Enrollment`
37       WHERE `SectionID` = p_section_id
38         AND `EnrollmentStatus` = 'ACTIVE';
39
40      IF v_active_count >= v_capacity THEN
41        SIGNAL SQLSTATE '45000'
42          SET MESSAGE_TEXT = 'Lop hoc phan da du si so, khong the ghi danh.';
43      END IF;
44    END IF;
45
46    INSERT INTO `Enrollment` (`StudentID`, `SectionID`, `EnrollmentDate`, `EnrollmentStatus`)
47    VALUES (p_student_id, p_section_id, CURRENT_TIMESTAMP, v_status);
48
49    SET v_new_id = LAST_INSERT_ID();
50    COMMIT;
```

D. Mã function

Listing 10: `fn_CalculateAssignmentCompletionRate`

```
1  USE `hcmut_lms`;
2
3  -- ============================================================================
4  -- BTL2 section 2.4: two stored functions.
5  -- Both use a cursor, IF/LOOP, input validation, a query, and calculation.
6  -- ============================================================================
7  DELIMITER $$
8
9  -- ----------------------------------------------------------------------------
10 -- fn_CalculateAssignmentCompletionRate
11 -- Completion = distinct ACTIVE enrolled students who submitted at least
12 -- one SUBMITTED attempt / all ACTIVE enrolled students * 100.
13 -- Returns 0.00 when there are no eligible students.
14 -- ----------------------------------------------------------------------------
15 DROP FUNCTION IF EXISTS `fn_CalculateAssignmentCompletionRate`$$
16 CREATE FUNCTION `fn_CalculateAssignmentCompletionRate`(
17   p_assignment_id INT
18 )
19 RETURNS DECIMAL(5,2)
20 READS SQL DATA
21 BEGIN
22   DECLARE v_section_id INT DEFAULT NULL;
```

```
23  DECLARE v_student_id INT DEFAULT NULL;
24  DECLARE v_total INT DEFAULT 0;
25  DECLARE v_submitted INT DEFAULT 0;
26  DECLARE v_has_submission INT DEFAULT 0;
27  DECLARE v_done BOOLEAN DEFAULT FALSE;
28
29  DECLARE cur_enrolled CURSOR FOR
30    SELECT e.`StudentID`
31      FROM `Enrollment` e
32     WHERE e.`SectionID` = v_section_id
33       AND e.`EnrollmentStatus` = 'ACTIVE'
34     ORDER BY e.`StudentID`;
35
36  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
37
38  IF p_assignment_id IS NULL OR p_assignment_id <= 0 THEN
39    SIGNAL SQLSTATE '45000'
40      SET MESSAGE_TEXT = 'AssignmentID phai la so nguyen duong.';
41  END IF;
42
43  SELECT MAX(a.`SectionID`) INTO v_section_id
44    FROM `Assignment` asg
45    JOIN `Assessment` a ON a.`AssessmentID` = asg.`AssessmentID`
46   WHERE asg.`AssessmentID` = p_assignment_id;
47
48  IF v_section_id IS NULL THEN
49    SIGNAL SQLSTATE '45000'
50      SET MESSAGE_TEXT = 'Khong tim thay bai tap can tinh ty le hoan thanh.';
51  END IF;
52
53  OPEN cur_enrolled;
54
55  enrollment_loop: LOOP
56    FETCH cur_enrolled INTO v_student_id;
57    IF v_done THEN
58      LEAVE enrollment_loop;
59    END IF;
60
61    SET v_total = v_total + 1;
62
63    SELECT COUNT(*) INTO v_has_submission
64      FROM `Submission` s
65     WHERE s.`AssignmentID` = p_assignment_id
66       AND s.`StudentID` = v_student_id
67       AND s.`SubmissionStatus` = 'SUBMITTED';
68
69    IF v_has_submission > 0 THEN
70      SET v_submitted = v_submitted + 1;
71    END IF;
72  END LOOP;
73
```

```
  CLOSE cur_enrolled;

  IF v_total = 0 THEN
    RETURN 0.00;
  END IF;

  RETURN ROUND(v_submitted * 100.0 / v_total, 2);
```

Listing 11: fn_CalculateStudentAverageScore

```
END$$

-- ----------------------------------------------------------------------
-- fn_CalculateStudentAverageScore
-- Internal analysis metric: arithmetic mean of published assessment
-- percentages. This is NOT the official HCMUT course-grade formula.
-- ----------------------------------------------------------------------
DROP FUNCTION IF EXISTS `fn_CalculateStudentAverageScore`$$
CREATE FUNCTION `fn_CalculateStudentAverageScore`(
  p_enrollment_id INT
)
RETURNS DECIMAL(5,2)
READS SQL DATA
BEGIN
  DECLARE v_exists INT DEFAULT 0;
  DECLARE v_grade_count INT DEFAULT 0;
  DECLARE v_percent DECIMAL(8,4) DEFAULT 0;
  DECLARE v_sum DECIMAL(12,4) DEFAULT 0;
  DECLARE v_done BOOLEAN DEFAULT FALSE;

  DECLARE cur_grades CURSOR FOR
    SELECT (g.`Score` * 100.0 / a.`MaxScore`)
      FROM `Grade` g
      JOIN `Assessment` a ON a.`AssessmentID` = g.`AssessmentID`
     WHERE g.`EnrollmentID` = p_enrollment_id
       AND g.`IsPublished` = TRUE
     ORDER BY g.`GradeID`;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

  IF p_enrollment_id IS NULL OR p_enrollment_id <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'EnrollmentID phai la so nguyen duong.';
  END IF;

  SELECT COUNT(*) INTO v_exists
    FROM `Enrollment`
   WHERE `EnrollmentID` = p_enrollment_id;

  IF v_exists = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Khong tim thay ghi danh can tinh diem trung binh.';
```

```
  END IF;

  OPEN cur_grades;

  grade_loop: LOOP
    FETCH cur_grades INTO v_percent;
    IF v_done THEN
      LEAVE grade_loop;
    END IF;

    SET v_sum = v_sum + v_percent;
    SET v_grade_count = v_grade_count + 1;
  END LOOP;

  CLOSE cur_grades;

  IF v_grade_count = 0 THEN
    RETURN 0.00;
  END IF;

  RETURN ROUND(v_sum / v_grade_count, 2);
END$$
```

E. Lệnh kiểm tra và checklist nộp bài

Listing 12: Chạy lại toàn bộ kiểm thử

```
cd hcmut-lms-btl2
uv run pytest -q                          # 72 passed
.venv/bin/python scripts/verify_database.py
.venv/bin/python scripts/smoke_api.py     # Flask phai dang chay
.venv/bin/python scripts/check_submission_readiness.py --with-tests
```

Checklist trước khi nộp:

1. Điền họ tên/MSSV vào bảng phân công và trang bìa.
2. Mỗi thành viên có commit gắn với SQL object mình sở hữu.
3. Chạy lại 4 lệnh trên máy sạch; lưu log.
4. Chạy `check_environment.bat` trên Windows thật nếu có.
5. Kiểm tra `.env` không bị commit.
6. Đính kèm PDF báo cáo và ảnh minh chứng nếu giảng viên yêu cầu.